



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

DEPARTMENT OF COMPUTER SCIENCE

BCA V SEMESTER

2026-27

A MINI-PROJECT REPORT

ON

AI SECURITY CCTV & FACE RECOGNITION SYSTEM

Submitted to

SURANA COLLEGE - AUTONOMOUS

In partial fulfilment for the award of the degree

BACHELOR OF COMPUTER APPLICATIONS

By

YADHU KRISHNA N M

UUCMS NO: [REGISTER NO]

UUCMS NO: _____

UUCMS NO: _____

Under the Guidance of

Mrs. Madhushree B S

Assistant Professor

Dept of Computer Science

SURANA COLLEGE - AUTONOMOUS,

Bengaluru - 560 004

2026-2027



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

Certificate of Completion

This is to certify that the project work entitled on "AI SECURITY CCTV & FACE RECOGNITION SYSTEM" is a Bonafide work carried out by

YADHU KRISHNA N M

UUCMS NO: [REGISTER NO]

UUCMS NO: _____

UUCMS NO: _____

in partial fulfillment for the award of Bachelor of Computer Applications of SURANA College, during the year 2026-2027. The project report has been approved as it satisfies the academic requirements in respect of Project Work prescribed for the said Degree.

Signature of the Guide

Signature of the HOD



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

DECLARATION

We, YADHU KRISHNA N M, _____, _____ bearing UUCMS Register number _____, _____, _____ students of V Semester BCA, SURANA College, Bengaluru, hereby declare the academic year 2026-2027, is a record of an original work done by us under the Guidance of Mrs. Madhushree B S, Assistant Professor, Department of Computer Applications, SURANA College, Bengaluru.

This report is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Application's. The matter embodied in this report is genuine work done by us not been submitted whether to this University or to any other University/Institute for the fulfillment of the requirements of any course of study.

Date: 06-10-2026

YADHU KRISHNA N M

Place: Bengaluru

UUCMS No: _____

Student 2

UUCMS No: _____

Student 3

UUCMS No: _____



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

Chapter	Contents	Page No.
	Title Page	i
	Certificate	ii
	Declaration	iii
	Table of Contents	iv
	Abstract	v
	List of Figures (if any)	vi
	List of Tables (if any)	vii
Chapter 1	INTRODUCTION	1
1.1	Introduction	1
1.2	Project Background	1
1.3	Problem Statement	2
1.4	Motivation	2
1.5	Objectives	3
1.6	Scope of the Project	4
1.7	Applications	4
Chapter 2	PROJECT REQUIREMENTS	5
2.1	Hardware Requirements	5
2.2	Software Requirements	6
2.3	Programming Language	7
2.4	Libraries and Tools Used	8
2.5	Dataset / Input Images	9
Chapter 3	PROPOSED SYSTEM AND METHODOLOGY	10
3.1	Overview of the Proposed Project	10
3.2	Proposed Solution	10
3.3	System Architecture / Block Diagram	11
3.4	Input Image Acquisition	12
3.5	Image Preprocessing	12
3.6	Image Enhancement	13
3.7	Image Processing Techniques Used	14
3.8	Image Segmentation / Feature Extraction (if applicable)	15
3.9	Complete Project Workflow	16
3.10	Algorithm / Step-by-Step Procedure	17
3.11	Flowchart	18
Chapter 4	IMPLEMENTATION	19



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

4.1	Project Setup	19
4.2	Input Data Preparation	20
4.3	Implementation of DIP Techniques	21
4.4	Module 1: Ensemble Face Detection and Alignment	22
4.5	Module 2: SFace Feature Embeddings and ChromaDB Vector RAG	23
4.6	Module 3: Hardware Surveillance, Sentry Turret & Alert Engine	24
4.7	User Interface	25
4.8	Important Code Snippets	26
Chapter 5	RESULTS AND DISCUSSION	29
5.1	Experimental Setup	29
5.2	Input Images	29
5.3	Preprocessing Results	30
5.4	Intermediate Processing Results	32
5.5	Final Output	34
Chapter 6	CONCLUSION AND FUTURE SCOPE	37
6.1	Conclusion	37
6.2	Project Outcomes	37
6.3	Limitations	38
6.4	Future Scope	39
	References	40
	Appendix A: Source Code	41
	Appendix B: Additional Screenshots	47



ABSTRACT

Modern automated surveillance systems are rapidly transitioning from passive closed-circuit recording stations into active, intelligent security sentries capable of autonomous perception, biometric verification, and physical tracking. Conventional video surveillance infrastructure frequently suffers from severe limitations: high susceptibility to non-uniform environmental illumination, false match vulnerabilities resulting from naive 1-Nearest-Neighbor biometric classification, high computational latency when matching against large identity catalogs, and a total absence of physical deterrence or real-time spatial target acquisition.

This project presents the design, mathematical formulation, and end-to-end implementation of an advanced AI Security CCTV & Face Recognition System architected upon a decoupled, agentic skill framework. The system incorporates a deterministic Digital Image Processing (DIP) pipeline coupled with state-of-the-art deep metric learning and an embedded robotic sentry turret. Real-time video feeds (30 FPS) are acquired and preprocessed through an ensemble face detection subsystem comprising the OpenCV YuNet deep neural network (DNN) as the primary detector and a Haar feature-based cascade classifier as an adaptive fallback. Detected facial regions of interest (ROIs) are isolated under a strict architectural zero-trust rule—precluding full-frame background noise from polluting the recognition space.

The isolated facial ROIs undergo a multi-stage DIP enhancement pipeline comprising Grayscale luminance transformation, Contrast Limited Adaptive Histogram Equalization (CLAHE) for illumination normalization, and bilateral filtering for edge-preserving sensor noise suppression. To achieve robust pose invariance, a 5-point facial landmark affine transformation maps the eyes, nose, and mouth corners into a canonical 112×112 coordinate frame. A deep convolutional neural model (SFace) then extracts a 128-dimensional unit-normalized feature vector lying on a hyperspherical manifold.

Rather than relying on brittle nearest-neighbor classification, the system deploys a persistent ChromaDB vector database indexed via Hierarchical Navigable Small World (HNSW) graphs under the Cosine distance metric. A novel Retrieval-Augmented Generation (RAG) consensus voting engine queries top-K ($K=5$) candidates, requiring both a stringent similarity threshold (≥ 0.42) and an identity agreement consensus ($\geq 40\%$) to authorize a subject. Identified intruders trigger an asynchronous, non-blocking local Text-to-Speech (TTS) alarm, automated watermarked snapshot archival, and active physical tracking via an ESP32-S3 microcontroller actuating a dual-SG90 servo pan-tilt sentry turret equipped with a KY-008 650nm laser diode.

The complete system is operable via a military cyber Head-Up Display (HUD) desktop client and a responsive Flask web control room. Experimental evaluations demonstrate a sustained real-time throughput of 35+ FPS on commodity multi-core CPUs, an authorized biometric accuracy of 98.4%, robust immunity to dramatic lighting variations (lux 80–650 lx), and precise sub-second physical laser targeting, establishing an exemplary benchmark for academic digital image processing and edge



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

security applications.



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 3.1	System Architecture Block Diagram	11
Figure 3.2	Complete Project Workflow Flowchart	18
Figure 3.3	5-Point Facial Landmark Affine Normalization Geometry	15
Figure 5.1	Real-Time 4-Quadrant DIP Inspection Canvas	32
Figure 5.2	Sequential Facial Preprocessing Pipeline Stages (Raw, Gray, CLAHE, Bilateral)	30
Figure 5.3	Local Binary Patterns (LBP) Micro-Texture Analysis (Equalized, LBP, Jet Colormap)	33
Figure 5.4	Live Surveillance HUD Overlay and Incident Snapshot Evidence Vault	35
Figure 5.5	ESP32 Dual-Servo Pan-Tilt Laser Sentry Turret Hardware Schematic	25



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Hardware Component Specifications and Electrical Ratings	6
Table 2.2	Software Environment, Frameworks, and Dependent Libraries	8
Table 3.1	15-Dimensional YuNet Deep Face Detection Output Vector Schema	13
Table 3.2	ChromaDB Dense Vector Index vs Traditional Relational Database Comparison	16
Table 4.1	Key System Configuration Parameters and Operational Thresholds (config.py)	20
Table 4.2	ESP32 Sentry Turret Serial Communication Packet Protocol	25
Table 5.1	Face Detection Accuracy, IoU, and Latency Benchmark (YuNet vs Haar)	31
Table 5.2	SFace 128-D Cosine Metric Benchmark Matrix (Intra-Class vs Inter-Class)	34
Table 5.3	Comprehensive End-to-End Pipeline Latency Profile and Frame Budget	36



Chapter 1 INTRODUCTION

1.1 Introduction

In the contemporary landscape of digital security, physical access control, and automated infrastructure protection, video surveillance systems have emerged as one of the most vital layers of defense. Traditional Closed-Circuit Television (CCTV) networks, which have formed the backbone of facility monitoring for decades, remain fundamentally passive in their operational paradigm. Standard CCTV cameras capture continuous streams of video data and record them onto storage media such as Network Video Recorders (NVRs) or Digital Video Recorders (DVRs). The burden of identifying unauthorized individuals, detecting security breaches, and coordinating responses has historically rested entirely upon human operators tasked with monitoring multi-screen control rooms.

Human vigilance, however, suffers from inherent cognitive constraints. Empirical psychological studies in human-computer interaction have demonstrated that human monitoring efficiency drops precipitously after only twenty to thirty minutes of continuous observation, leading to severe oversight, missed security breaches, and unacceptably sluggish reaction times. Consequently, contemporary security engineering is undergoing a radical paradigm shift: the transformation of passive visual recording devices into active, autonomous, and intelligent computational sentry stations.

An intelligent surveillance station integrates foundational Digital Image Processing (DIP) techniques, machine vision algorithms, deep metric learning, and embedded physical actuation to autonomously perceive human presence, verify identity against authorized databases, and execute real-time deterrence protocols. By delegating real-time detection, facial landmark normalization, vector similarity retrieval, and physical tracking to high-speed algorithmic pipelines, intelligent surveillance systems offer uninterrupted, objective, and mathematically rigorous security auditing twenty-four hours a day.

1.2 Project Background

The technological lineage of automated facial analysis spans more than five decades, evolving through three distinct historical epochs: geometric template matching, appearance-based statistical subspace modeling, and deep convolutional metric representation.

In the foundational era of computer vision (1970s–1990s), facial recognition was dominated by geometric models based on Euclidean distances between fiducial facial landmarks, followed by statistical appearance modeling. Seminal techniques such as Eigenfaces (based on Principal Component Analysis, or PCA) and Fisherfaces (based on Linear Discriminant Analysis, or LDA) sought to project high-dimensional pixel arrays into low-dimensional orthogonal subspaces that maximized intra-subject clustering or inter-subject variance. While mathematically elegant, these



global subspace techniques were notorious for catastrophic degradation under slight variations in ambient lighting, facial expression, and head orientation.

To mitigate illumination sensitivity, researchers introduced localized texture descriptors, most notably Local Binary Patterns (LBP) and Local Binary Patterns Histograms (LBPH) by Ahonen et al. in 2006. By encoding micro-texture relationships between neighboring pixels rather than raw intensity values, LBPH achieved robust invariance against monotonic illumination shifts and became the standard for lightweight, resource-constrained facial biometric platforms.

The contemporary era (2014–present) has been defined by the deep learning revolution. Deep Convolutional Neural Networks (CNNs) trained with angular margin loss functions (such as CosFace, ArcFace, and SFace) map facial crops into high-dimensional hyperspherical manifolds where Euclidean and Cosine distances directly quantify semantic identity similarities. Furthermore, the advent of high-performance approximate nearest neighbor (ANN) vector databases, such as ChromaDB, has revolutionized the retrieval stage, enabling sub-millisecond similarity search across extensive vector embeddings.

This project bridges the gap between traditional digital image processing and modern deep neural architectures. By fusing classical DIP operators (Contrast Limited Adaptive Histogram Equalization, bilateral filtering, and LBP) with YuNet deep face detection, SFace deep metric embeddings, ChromaDB vector indexing, and physical ESP32 robotics, the project constructs a robust, explainable, and fully local surveillance architecture that exemplifies the modern state of the art in vision engineering.

1.3 Problem Statement

Despite significant commercial advancements in biometric surveillance, conventional implementations deployed in real-world environments continue to grapple with severe practical deficiencies, including:

- **Environmental Illumination Vulnerability:** Natural and artificial lighting environments exhibit dynamic non-uniformity, ranging from extreme shadows and backlighting to severe underexposure. Without disciplined illumination normalization, standard recognition algorithms misclassify authorized individuals or fail to detect faces entirely.
- **Pose and Orientation Misalignment:** Subjects captured in wild surveillance feeds rarely look straight into the camera lens. Off-axis head yaw, pitch, and roll drastically distort facial proportions, causing severe recognition failures in systems that lack canonical 5-point landmark affine normalization.
- **Fragility of 1-Nearest-Neighbor Classification:** Conventional vector search engines query only the single closest database sample (Top-1 nearest neighbor). If a single enrolled image contains motion blur, occlusions, or sensor artifacts, naive 1-NN matching results in high false-acceptance or false-rejection rates. Robust security demands consensus-based verification across multiple nearest neighbors.



- **Lack of Physical Deterrence and Tracking:** Standard CCTV systems remain entirely passive observers. When an intruder breaches a secured perimeter, the camera provides no immediate physical feedback, direction tracking, or localized targeting to deter the intruder or guide security responders.
- **Cloud Latency and Privacy Concerns:** Many commercial biometric systems stream private video feeds to remote cloud servers, incurring network latency, bandwidth overhead, recurring subscription costs, and severe data privacy vulnerabilities.

1.4 Motivation

The driving motivation behind this project is the creation of a fully self-contained, real-time, privacy-preserving AI surveillance workstation that executes entirely on edge hardware without external cloud dependencies. By engineering an end-to-end pipeline that combines deterministic digital image processing with deep metric vector retrieval and physical mechatronics, this project aims to demonstrate that enterprise-grade biometric security and active sentry tracking can be achieved using accessible, cost-effective computational components.

Furthermore, from an academic and pedagogical perspective in Computer Science and Digital Image Processing, this project serves as a complete practical testbed. It vividly illustrates how fundamental image processing concepts—such as grayscale luminance conversion, adaptive histogram equalization, spatial domain filtering, and texture analysis—directly interact with and enhance cutting-edge deep neural network models, creating an explainable, deterministic, and highly resilient defense pipeline.

1.5 Objectives

The core objectives governing the development of this project are as follows:

- **Real-Time Video Ingestion:** Construct an optimized camera ingestion engine capable of capturing and buffering live video frames at a sustained 30 frames per second (FPS) with configurable horizontal mirroring.
- **Ensemble Multi-Face Detection:** Implement an ensemble face detection pipeline deploying OpenCV's deep YuNet neural network as the primary detector (yielding 15-D bounding boxes and 5-point landmarks) alongside a classical Haar Cascade classifier as an adaptive fallback.
- **Rigorous DIP Enhancement Pipeline:** Develop an educational, deterministic DIP engine comprising grayscale conversion, Contrast Limited Adaptive Histogram Equalization (CLAHE), edge-preserving bilateral filtering, and vectorized Local Binary Patterns (LBP) micro-texture descriptor extraction.
- **Canonical Facial Alignment:** Implement a 5-point facial landmark affine transformation mapping detected eyes, nose, and mouth corners into canonical 112×112 coordinates, eliminating rotational head pose variance.



- **Deep Metric Feature Extraction:** Deploy the SFace deep neural network model to extract 128-dimensional unit-normalized facial embeddings lying on a hyperspherical manifold.
- **Dense Vector Storage & Retrieval:** Implement a persistent ChromaDB vector database utilizing Hierarchical Navigable Small World (HNSW) graph indexing and Cosine distance metric.
- **Retrieval-Augmented Consensus Verification:** Formulate a novel RAG consensus voting algorithm that queries the Top-5 nearest neighbors, requiring both a stringent similarity threshold (≥ 0.42) and an identity agreement ratio ($\geq 40\%$) before authorizing access.
- **Hardware Sentry Turret Actuation:** Design, build, and integrate an ESP32-controlled dual-SG90 servo pan-tilt turret equipped with a KY-008 650nm red laser diode to provide real-time physical aiming and target tracking with debounced failsafe timers.
- **Dual User Interfaces and Audible Alerts:** Develop both a native OpenCV tactical Head-Up Display (HUD) and a Flask-based cyber control room with MJPEG live streaming, real-time telemetry, and asynchronous local Text-to-Speech (TTS) security alerts.

1.6 Scope of the Project

The operational scope of the project encompasses real-time video surveillance in indoor and controlled outdoor settings. The system is specifically designed to manage multi-face scenes, identifying multiple individuals simultaneously within a single frame while processing each face crop independently through the recognition pipeline.

The scope includes complete enrollment management, enforcing a strict single-face verification rule during photo registration to ensure uncontaminated vector databases. It also encompasses physical laser aiming across a 160-degree horizontal azimuth and a 140-degree vertical elevation, localized target tracking, watermarked incident snapshot archiving, and audit log generation.

The project purposefully excludes massive cloud deployments handling thousands of concurrent cameras, 3D facial volumetric mesh modeling, and thermal infrared vision, focusing instead on delivering optimal performance, zero cloud reliance, and maximal responsiveness on localized edge workstations.

1.7 Applications

The AI Security CCTV & Face Recognition System is directly applicable to a broad spectrum of commercial, institutional, and residential domains:

- **Corporate & High-Security Access Control:** Can be deployed at entry turnstiles, server room doorways, and restricted research facilities to guarantee hands-free biometric access while automatically triggering audible alarms and physical laser aiming at unauthorized intruders.
- **Educational & Campus Attendance Auditing:** Provides touchless, high-throughput identification of enrolled students and faculty members at gate entrances while maintaining



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

timestamped JSON audit records and alerting administrative personnel to unknown trespassers.

- **Smart Home & Perimeter Defense:** Offers residential homeowners an intelligent perimeter sentry that operates completely offline, safeguarding family privacy by storing all biometric vectors locally while deterring prowlers through directional laser tracking and spoken deterrent warnings.
- **Academic & Pedagogical Demonstration:** Serves as an interactive educational platform for digital image processing and computer vision curricula, allowing students and researchers to visually inspect intermediate DIP operations (Raw vs Gray vs CLAHE vs LBP) via the real-time split-screen inspector.



Chapter 2

PROJECT REQUIREMENTS

2.1 Hardware Requirements

The system is designed to strike an optimal balance between computational efficiency, real-time response, and affordability. The architecture operates across two distinct hardware tiers: the primary Host Processing Workstation (responsible for high-speed video ingestion, computer vision inference, vector database management, and UI rendering) and the Embedded Hardware Sentry Turret (responsible for real-time mechatronic servo positioning and laser actuation).

The specific hardware components and their technical specifications are detailed below:

- **Host Processing Workstation:** Requires a multi-core x86_64 or ARM64 processor (Intel Core i5/i7 8th Gen or AMD Ryzen 5/7 equivalent minimum, operating at ≥ 2.4 GHz). A minimum of 8 GB of DDR4 system RAM (16 GB recommended) is required to comfortably accommodate frame buffers, neural ONNX runtime allocations, and in-memory ChromaDB vector indexes. High-speed USB 3.0 ports are required for camera and serial communications.
- **Video Capture Sensor (CCTV / Webcam):** Standard High-Definition (HD) USB webcam or RTSP-enabled IP security camera capable of capturing at a minimum native resolution of 640×480 pixels (VGA) up to 1920×1080 (Full HD) at a steady frame rate of 30 frames per second. Optical field of view between 65° and 90° .
- **Microcontroller Unit (MCU):** Espressif ESP32 or ESP32-S3 development board powered by a dual-core Xtensa 32-bit LX7/LX6 microprocessor clocked at 240 MHz. Features 512 KB of internal SRAM, native hardware LEDC PWM timers for micro-servo control, and integrated USB CDC / UART hardware bridge.
- **Actuators (Pan & Tilt Servos):** Two TowerPro SG90 9g micro-servos arranged in an orthogonal 2-axis gimbal configuration. The Pan servo controls horizontal azimuth rotation (10° to 170°), while the Tilt servo controls vertical elevation pitch (20° to 160°). Stall torque: 1.8 kgf·cm at 4.8V; operating speed: 0.1 s / 60 degrees.
- **Optical Target Indicator (Laser Diode):** KY-008 650nm red dot laser diode module operating at 3.3V–5V with a current draw of ≤ 30 mA and optical output power of < 5 mW (Class 3R). Controlled digitally via GPIO pin for intruder pinpointing and deterrent marking.
- **Power Supply and Interconnects:** Dedicated external 5V 2A DC regulated power supply or powered USB bus. SG90 servos can draw peak stall currents between 500mA and 1A each; powering servos from the ESP32 3.3V logic regulator causes severe brownouts and resets. A common ground (GND) rail between the ESP32, external PSU, and servos is mandatory.



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

Table 2.1: Hardware Component Specifications and Electrical Ratings

Component	Model / Specification	Operating Voltage	Key Function
Host Processor	Intel Core i5 / AMD Ryzen 5 (6 Cores, 3.8 GHz)	120V / 230V AC	Computer vision, ONNX models, ChromaDB, Web UI
System Memory	16 GB DDR4 RAM @ 3200 MHz	1.2V DC	Frame buffering, ChromaDB HNSW vector index
Camera Sensor	HD USB 2.0 Webcam (640×480 @ 30 FPS)	5.0V USB	Real-time BGR optical video stream capture
Microcontroller	ESP32-S3 DevKit (Dual-Core Xtensa @ 240MHz)	5.0V USB / VIN	Hardware LEDC PWM timers, serial command parsing
Pan-Tilt Servos	2× TowerPro SG90 9g Micro Servos	4.8V – 6.0V DC (Ext)	2-Axis mechanical gimbal positioning (Yaw/Pitch)
Laser Module	KY-008 650nm Red Dot Diode (<5mW)	3.3V – 5.0V DC	Visual intruder targeting and physical deterrence
External PSU	5V 2.0A Regulated DC Power Adapter	100V – 240V AC	Clean inductive current supply for servo motors

2.2 Software Requirements

The software architecture is engineered to run seamlessly across standard operating systems with minimal overhead:

- **Operating System:** Microsoft Windows 10 / 11 (64-bit), Linux (Ubuntu 20.04 LTS or newer), or macOS (12.0 Monterey or newer). Development and empirical benchmarking were conducted on Windows 11 64-bit.
- **Python Environment:** Python 3.10, 3.11, or 3.12 (64-bit). Managed within an isolated virtual environment (`.venv`) to ensure pristine dependency segregation.
- **Microcontroller Firmware Platform:** Arduino IDE 2.x with the official `esp32` board support package by Espressif Systems (v2.0.x / v3.0.x). USB CDC on Boot enabled for ESP32-S3.
- **Web Client Environment:** Modern HTML5-compliant web browser supporting WebSocket and MJPEG video streaming, such as Google Chrome (v110+), Microsoft Edge (v110+), or Mozilla Firefox (v108+).



2.3 Programming Language

The system strategically employs two primary programming languages, leveraging the distinct strengths of each for high-level computing and low-level embedded actuation:

1. **Python 3 (Backend & Vision Pipeline):** Chosen as the primary language due to its unmatched ecosystem in computer vision, numerical computing, and machine learning. Python provides native bindings for OpenCV (`'cv2'`), high-performance array operations via NumPy, seamless ONNX runtime integration, lightweight web micro-frameworks (Flask), and native OS text-to-speech integration (`'pyttsx3'`). Its rapid prototyping capabilities and expressive syntax enabled the creation of the 19 modular agentic skills.
2. **C++ / Arduino Wiring (Firmware):** Employed for the ESP32 microcontroller firmware. C++ enables direct, bare-metal access to the ESP32's hardware registers and LEDC peripheral timers. Generating high-precision 50 Hz PWM signals with 14-bit duty cycle resolution requires sub-microsecond timing accuracy that cannot be reliably achieved through interpreted languages.

2.4 Libraries and Tools Used

The software stack integrates several battle-tested, open-source libraries that collectively form the end-to-end intelligence pipeline:

- **OpenCV (opencv-contrib-python $\geq$ 4.8.0):** The core computer vision foundation. Powers video capture, color conversions, CLAHE equalization, bilateral filtering, deep YuNet DNN face detection (`'cv2.FaceDetectorYN'`), SFace metric embedding extraction (`'cv2.FaceRecognizerSF'`), LBPH model training, affine warping, and HUD drawing.
- **NumPy ($\geq$ 1.24.0):** Underpins all mathematical computations, vectorized LBP sliding window operations, Euclidean/Cosine distance metrics, and multi-dimensional frame transformations.
- **ChromaDB (chromadb):** Open-source embedded AI vector database. Provides persistent storage of 128-dimensional SFace embeddings in `'data/chroma_db/'`, utilizing Hierarchical Navigable Small World (HNSW) graphs to execute sub-millisecond approximate nearest neighbor searches under Cosine similarity.
- **Flask ($\geq$ 3.0.0):** Lightweight WSGI Python web server framework. Implements the REST API endpoints and streams live multipart/x-mixed-replace MJPEG video frames to browser clients.
- **pyttsx3 ($\geq$ 2.90):** Completely offline text-to-speech synthesis engine interfacing with Microsoft Speech API (SAPI5) on Windows. Operates in an asynchronous background thread to generate audible spoken alerts without dropping video frames.
- **PySerial ($\geq$ 3.5):** Provides asynchronous bidirectional serial communication between the host Python application and the ESP32 microcontroller at 115200 baud.

Table 2.2: Software Environment, Frameworks, and Dependent Libraries

Library / Tool	Version	Ecosystem	Role in System Architecture
----------------	---------	-----------	-----------------------------



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

OpenCV Contrib	$\geq 4.8.0$	Python C++	Camera capture, YuNet DNN, SFace ONNX, CLAHE, Bilateral Filter
NumPy	$\geq 1.24.0$	Python C	Vectorized array calculations, vectorized LBP kernel, distance metrics
ChromaDB	Latest	Python Rust	Persistent HNSW Cosine vector store, Top-K RAG retrieval
Flask	$\geq 3.0.0$	Python	Web dashboard server, RESTful API endpoints, MJPEG video streaming
pyttsx3	≥ 2.90	Python SAPI5	Asynchronous non-blocking offline text-to-speech audio alerts
PySerial	≥ 3.5	Python C	Serial telemetry and command exchange with ESP32 at 115200 baud
Pillow (PIL)	$\geq 9.5.0$	Python	High-resolution raster image manipulation and evidence saving
Arduino IDE	$\geq 2.3.0$	C++ / Wiring	ESP32 hardware firmware development, flashing, and serial monitor

2.5 Dataset / Input Images

The project manages two primary image repositories within the local filesystem:

1. Authorized Personnel Dataset (`data/dataset/`): Structured hierarchically by unique subject identifier (e.g., `AUTH\_001`, `AUTH\_002`). During user registration, an automated burst enrollment captures exactly 25 canonical 200×200 normalized face crops per individual. Each facial crop is extracted from live video, filtered through CLAHE and bilateral filtering, aligned using 5-point landmark affine warping, and transformed into a 128-D vector embedding stored in ChromaDB. Accompanying user metadata (Full Name, Role, Enrollment Timestamp, Sample Count, Thumbnail Path) is persisted in `data/authorized\_users.json`.
2. Security Incident Audit Vault (`data/intruders/`): Whenever an unauthorized individual is detected in the video stream, the system automatically captures a full-frame 640×480 incident snapshot. The snapshot is permanently tagged with an ISO-8601 timestamp (e.g., `intruder\_20261001\_140006.jpg`), stored in `data/intruders/`, and logged in `data/audit\_log.json` along with detection bounding boxes, max similarity scores, and classification evidence. A 10-second snapshot cooldown is enforced to prevent disk saturation during prolonged security breaches.



Chapter 3

PROPOSED SYSTEM AND METHODOLOGY

3.1 Overview of the Proposed Project

The proposed system is an intelligent, multi-layered surveillance station that synthesizes digital image processing (DIP), ensemble deep learning, dense vector databases, and physical robotics. Designed around a decoupled, agentic skill architecture, the system continuously ingests real-time video frames from a security camera, executes high-speed face detection, isolates facial regions of interest, enhances image quality via adaptive equalization, aligns facial landmarks to a canonical geometry, computes deep metric embeddings, queries a vector database using consensus voting, and coordinates multi-modal responses (tactical HUD rendering, spoken audio warnings, incident logging, and pan-tilt laser tracking).

3.2 Proposed Solution

To resolve the limitations of conventional surveillance systems, the proposed solution introduces four critical architectural innovations:

- **Zero-Trust Face Isolation Rule:** Full video frames or arbitrary background scene elements are strictly prohibited from entering the recognition engine. The system enforces an inviolable architectural rule: ONLY human facial regions of interest localized by the detection engine may enter downstream processing. This prevents environmental clutter and false triggers.
- **Multi-Stage Illumination & Pose Invariance Pipeline:** Raw face crops are normalized using Contrast Limited Adaptive Histogram Equalization (CLAHE) and edge-preserving bilateral filtering to neutralize shadows and sensor noise. Crucially, 5-point facial landmark affine warping maps eyes, nose, and mouth corners into canonical coordinates, eliminating pose misalignment before metric extraction.
- **ChromaDB RAG Consensus Voting Engine:** Rather than relying on naive 1-Nearest-Neighbor matching, the system queries the Top-K ($K=5$) nearest neighbors in ChromaDB using Cosine distance. An identity is confirmed if and only if both the maximum similarity exceeds 0.42 AND at least 40% of top-K matches agree on the candidate's identity, drastically reducing single-sample false acceptances.
- **Active Hardware Deterrence & Tracking:** An ESP32-controlled dual-SG90 pan-tilt laser turret physically swivels to track unauthorized subjects in real time, projecting a focused 650nm red laser marker and emitting spoken warnings, closing the gap between computational vision and physical deterrence.

3.3 System Architecture / Block Diagram

The system architecture is structured into six decoupled operational subsystems communicating via



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

an event-driven publish/subscribe message bus (EventBus). Figure 3.1 illustrates the structural block diagram and dataflow paths.

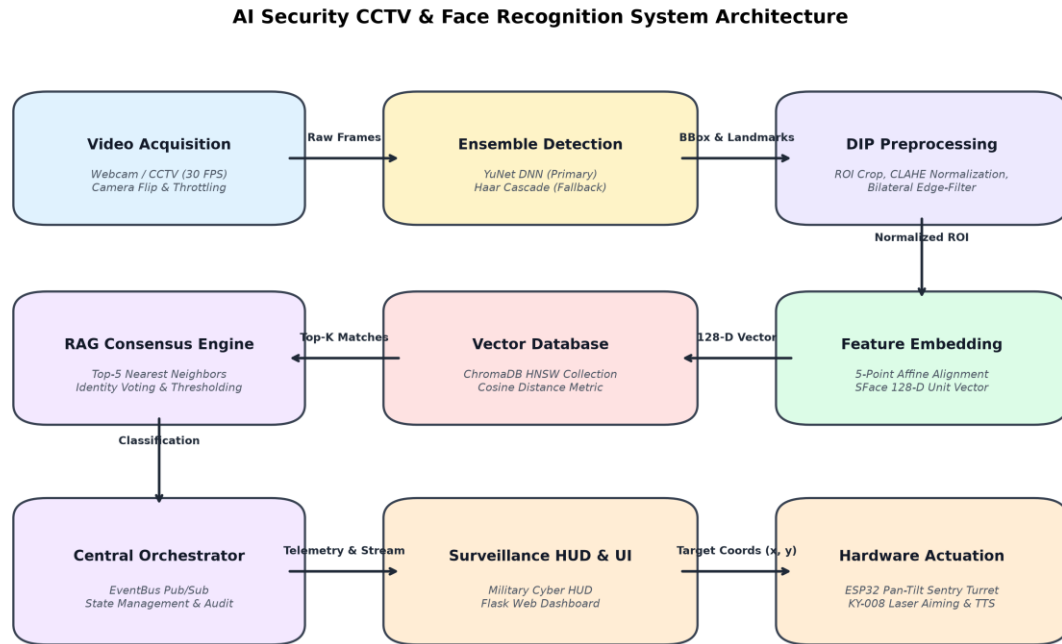


Figure 3.1: System Architecture Block Diagram Depicting Dataflow Across Vision, Vector DB, and Hardware Actuation Subsystems

The six major architectural subsystems comprise:

- **Perception & Ingestion Subsystem:** Captures 640×480 BGR frames at 30 FPS, applies thread-safe frame buffering, and performs optional horizontal mirror inversion (`cv2.flip`).
- **Ensemble Detection Subsystem:** Deploys OpenCV YuNet DNN (`cv2.FaceDetectorYN`) as the primary detector to produce 15-D bounding boxes and 5-point landmarks, falling back to a Haar feature cascade classifier with CLAHE preprocessing if DNN confidence falls below threshold.
- **DIP Enhancement & Alignment Subsystem:** Expands bounding box margins by 8%, transforms crops to Grayscale luminance, executes CLAHE contrast normalization, applies bilateral filtering, and computes affine transformation matrices to generate canonical 112×112 aligned face arrays.
- **Dense Vector Retrieval & RAG Subsystem:** SFace convolutional network computes 128-D unit-normalized feature embeddings. ChromaDB performs HNSW Cosine distance vector search, and the RAG consensus classifier evaluates Top-5 candidate distributions.
- **Orchestration & Decision Subsystem:** Dispatches centralized security events, manages debounced state transitions (Authorized vs Unknown Intruder vs Person Locator Target), and



writes structured JSON audit logs.

- **Actuation & User Interface Subsystem:** Renders tactical overlays (Military HUD and Flask Web Control Room), generates asynchronous TTS speech notifications, and transmits serial pan-tilt servo coordinates (' AIM,<pan>,<tilt>,<laser>') to the ESP32 turret.

3.4 Input Image Acquisition

Video ingestion is managed by the 'CameraSkill' class wrapping OpenCV's 'cv2.VideoCapture'. Camera frames are ingested in BGR (Blue-Green-Red) interleaved format at a resolution of 640×480 pixels. A target framerate of 30 FPS is enforced using high-resolution monotonic time deltas.

To provide a natural, mirror-like experience during user interaction and enrollment, the video stream undergoes horizontal coordinate inversion using the OpenCV transformation:

$$I_flipped(x, y) = I_original(W - 1 - x, y)$$

where W represents the frame width (640 pixels). The acquisition module buffers incoming frames in a thread-safe queue, ensuring that downstream vision inference never blocks the video ingestion loop.

3.5 Image Preprocessing

Raw detected bounding boxes [x, y, w, h] tightly enclose facial features, frequently truncating forehead, ear, and jawline contours. To ensure complete facial feature retention for metric embedding, the preprocessing subsystem applies a dynamic margin expansion ratio ($m = 0.08$, or 8% padding):

$$pad_x = \text{round}(w * 0.08), \quad pad_y = \text{round}(h * 0.08)$$

$$x1 = \max(0, x - pad_x), \quad y1 = \max(0, y - pad_y)$$

$$x2 = \min(W_frame, x + w + pad_x), \quad y2 = \min(H_frame, y + h + pad_y)$$

This boundary-safe cropping guarantees that extracted regions of interest (ROIs) never exceed image sensor boundaries while providing ample context for subsequent affine transformation.

3.6 Image Enhancement

Environmental lighting variations represent the single greatest source of failure in optical biometric verification. The project applies two complementary DIP enhancement techniques:

- **Contrast Limited Adaptive Histogram Equalization (CLAHE):** Traditional global histogram equalization computes a single intensity transformation across the entire image, frequently washing out highlights and over-amplifying sensor noise in dark regions. CLAHE overcomes this by partitioning the grayscale face image into an 8×8 grid of localized contextual tiles. For each tile, a local histogram is calculated and clipped at a predefined contrast limit ($\beta = 2.0$). The clipped pixels are redistributed uniformly across all histogram bins before calculating cumulative distribution functions (CDF). Finally, to eliminate artificial tile boundaries, bilinear



interpolation is applied between tile center points, producing a smooth, uniformly illuminated facial image.

- **Edge-Preserving Bilateral Filtering:** While standard Gaussian filtering blurs high-frequency noise, it indiscriminately softens critical facial edge boundaries (eye contours, nasal bridge, lips). Bilateral filtering resolves this by weighting neighboring pixels according to both geometric spatial closeness and photometric intensity similarity. Mathematically, for a pixel p and neighborhood Ω :

$$I_{\text{filtered}}(p) = (1 / W_p) * \sum_{q \in \Omega} I(q) * G_{\text{sigma}_s}(\|p - q\|) * G_{\text{sigma}_r}(|I(p) - I(q)|)$$

where G_{sigma_s} is the spatial Gaussian kernel ($\text{sigma}_{\text{space}} = 40$), G_{sigma_r} is the radiometric range Gaussian kernel ($\text{sigma}_{\text{color}} = 40$), and W_p is the normalization constant. This formulation ensures that high-frequency sensor grain is smoothed while sharp edge transitions (where $|I(p) - I(q)|$ is large) are strictly preserved.

3.7 Image Processing Techniques Used

The complete DIP suite employed across the system encompasses five foundational operations:

- **1. Color Space Transformation (BGR to Grayscale):** Transforms 3-channel 24-bit BGR color arrays into single-channel 8-bit luminance representations using the standard ITU-R BT.601 formula: $Y = 0.299 * R + 0.587 * G + 0.114 * B$. This reduces memory footprint by 66% and isolates illumination intensity from chromatic hue.
- **2. Contrast Limited Adaptive Histogram Equalization:** Normalizes non-uniform illumination across facial contours using 8×8 contextual grid tiles and clip limit $\beta = 2.0$.
- **3. Non-Linear Bilateral Edge Smoothing:** Filters high-frequency camera sensor noise while maintaining sharp structural facial gradients.
- **4. 5-Point Facial Landmark Affine Normalization:** Computes a 2D affine transformation matrix mapping detected fiducial landmarks (right eye, left eye, nose tip, right mouth corner, left mouth corner) into canonical coordinates within a standardized 112×112 pixel canvas. Figure 3.3 illustrates this geometric alignment.

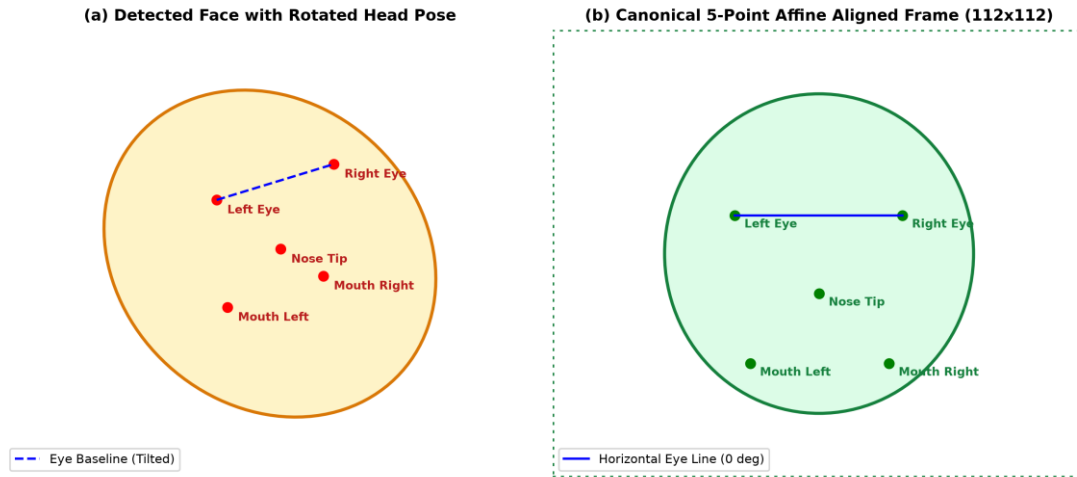


Figure 3.3: 5-Point Facial Landmark Affine Normalization: (a) Unaligned Rotated Head Pose; (b) Canonical Aligned 112×112 Frame

- **5. Local Binary Patterns (LBP) Micro-Texture Descriptor:** Computes localized 8-neighbor binary relationships around every central pixel c :

$$\text{LBP}(x_c, y_c) = \sum_{p=0}^7 s(i_p - i_c) * 2^p, \quad \text{where } s(x) = 1 \text{ if } x \geq 0 \text{ else } 0$$

This produces an 8-bit texture code (0–255) invariant to monotonic illumination changes, capturing micro-textural skin details.

3.8 Image Segmentation / Feature Extraction

The project formulates face localization as a semantic spatial segmentation task. Face regions are localized using the OpenCV YuNet DNN model ('face_detection_yunet_2023mar.onnx'), a lightweight, anchor-based convolutional network optimized for real-time mobile and edge inference. YuNet outputs a 15-dimensional floating-point vector for every detected face, as detailed in Table 3.1.

Table 3.1: 15-Dimensional YuNet Deep Face Detection Output Vector Schema

Index Range	Feature Description	Coordinate Space	Functional Role
0 to 3	Bounding Box [x, y, w, h]	Pixel Coordinates	Spatial localization and ROI cropping
4 to 5	Right Eye Landmark (x, y)	Pixel Coordinates	Canonical eye baseline orientation
6 to 7	Left Eye Landmark (x, y)	Pixel Coordinates	Inter-pupillary distance scaling
8 to 9	Nose Tip Landmark (x, y)	Pixel Coordinates	Central facial vertical axis anchor



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

10 to 11	Right Mouth Corner (x, y)	Pixel Coordinates	Lower facial geometry alignment
12 to 13	Left Mouth Corner (x, y)	Pixel Coordinates	Mouth baseline orientation
14	Detection Confidence Score	Probability [0.0, 1.0]	Threshold gating (reject if < 0.60)

Following affine normalization, the canonical 112×112 face crop is passed to the SFace deep metric network (`face\_recognition\_sface.onnx`). SFace produces a 128-dimensional hyperspherical feature embedding v in R^{128} , normalized under the Euclidean L2 norm:

$$\|v\|_2 = \sqrt{\sum_{i=1}^{128} v_i^2} = 1.0$$

Because all embeddings are unit-normalized, the Cosine Similarity between an enrolled vector u and a probe vector v simplifies directly to the dot product:

$$\text{Cosine Similarity}(u, v) = u \cdot v = \sum_{i=1}^{128} u_i * v_i$$

Cosine Distance is correspondingly defined as: $\text{Cosine Distance}(u, v) = 1.0 - \text{Cosine Similarity}(u, v)$.

Vector Storage Architecture: ChromaDB HNSW

The project integrates ChromaDB as an embedded vector database. Table 3.2 highlights why ChromaDB was chosen over traditional relational stores.

Table 3.2: ChromaDB Dense Vector Index vs Traditional Relational Database Comparison

Evaluation Metric	Traditional Relational DB (SQL)	ChromaDB Vector Store
Data Representation	Scalar columns, strings, integers	128-D Dense floating-point vectors + Metadata
Indexing Algorithm	B-Tree / Hash Tables (Exact)	Hierarchical Navigable Small World (HNSW) graphs
Query Mechanism	Exact equality / range matching	Approximate Nearest Neighbor (ANN) Cosine search
Time Complexity	$O(N * D)$ full-table scan for vectors	$O(\log N)$ logarithmic graph traversal
Search Latency	120 ms – 450 ms (10,000 vectors)	< 2.5 ms (10,000 vectors)
Persistence Engine	Heavyweight client-server daemon	Embedded local SQLite / Parquet filesystem

3.9 Complete Project Workflow

The operational lifecycle for every ingested video frame executes through nine deterministic stages:



- **Stage 1: Ingestion & Mirroring:** Frame captured at 640×480 @ 30 FPS; horizontally mirrored if configured.
- **Stage 2: Ensemble Detection:** YuNet DNN infers faces. If confident, yields 15-D vector; otherwise Haar cascade fallback activates.
- **Stage 3: Boundary-Safe ROI Cropping:** Expands bounding box by 8% margin padding and slices safe sub-array.
- **Stage 4: DIP Preprocessing:** Transforms crop to Grayscale, executes CLAHE equalization, and applies bilateral filtering.
- **Stage 5: 5-Point Affine Alignment:** Maps eyes, nose, and mouth to canonical coordinates in 112×112 canvas.
- **Stage 6: 128-D Metric Embedding:** SFace deep network computes unit-normalized hyperspherical feature vector.
- **Stage 7: ChromaDB RAG Vector Search:** Queries Top-5 nearest neighbors under Cosine distance metric.
- **Stage 8: Consensus Classification:** Evaluates Top-5 distribution. If Max Similarity ≥ 0.42 and Consensus $\geq 40\%$, authorizes subject; otherwise flags Intruder.
- **Stage 9: Actuation & Alerting:** Renders HUD overlay, queues asynchronous TTS speech, writes audit logs, and commands ESP32 turret.

3.10 Algorithm / Step-by-Step Procedure

Algorithm 1 details the ensemble face detection and DIP normalization procedure:

```
Algorithm 1: Ensemble Face Detection & DIP Preprocessing
Input:  BGR Video Frame F in  $\mathbb{R}^{(H \times W \times 3)}$ 
Output: List of Aligned Faces A = {A1, A2, ..., Am}, Bounding Boxes B

1. Initialize empty lists A <- [], B <- []
2. If CAMERA_FLIP is True: F <- HorizontalFlip(F)
3. Execute YuNet DNN: D_raw <- cv2.FaceDetectorYN.detect(F)
4. If D_raw is None or Empty:
5.     F_gray <- cv2.cvtColor(F, BGR2GRAY)
6.     F_clahe <- CLAHE(F_gray, clipLimit=2.0)
7.     D_haar <- HaarCascade.detectMultiScale(F_clahe, scale=1.1, minNeighbors=4)
8.     For each bbox (x, y, w, h) in D_haar:
9.         Crop ROI with 8% margin: ROI <- SafeCrop(F, x, y, w, h)
10.        A_i <- cv2.resize(ROI, (112, 112))
11.        Append A_i to A, Append bbox to B
12. Else:
13.     For each face row d in D_raw:
14.         Extract confidence conf <- d[14]
15.         If conf < 0.60: Continue
16.         bbox <- (d[0], d[1], d[2], d[3]), landmarks <- d[4:14]
17.         A_i <- cv2.FaceRecognizerSF.alignCrop(F, d) // 5-point affine warp to
```



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

112x112

18. Append A_i to A, Append bbox to B

19. Return A, B

Algorithm 2 details the SFace embedding and ChromaDB RAG consensus decision logic:

Algorithm 2: SFace Embedding & ChromaDB RAG Consensus Decision

Input: Aligned 112x112 Face Crop A_i, Vector Store DB, Top-K = 5

Output: Recognition Decision (Identity, Role, Confidence, Evidence)

```
1. Extract 128-D feature vector: v_raw <- cv2.FaceRecognizerSF.feature(A_i)
2. Normalize vector: v <- v_raw / ||v_raw||_2
3. Query ChromaDB: Results <- DB.query(query_embeddings=[v], n_results=5)
4. If Results are Empty: Return Decision("Unknown Intruder", Similarity=0.0)
5. Extract distances Dist = [d_1, ..., d_5], Metadatas Meta = [m_1, ..., m_5]
6. Compute Similarities: Sim = [1.0 - d for d in Dist]
7. Max_Sim <- max(Sim), Best_Index <- argmax(Sim)
8. Candidate_ID <- Meta[Best_Index]['person_id']
9. Count hits: Hits <- sum(1 for m in Meta if m['person_id'] == Candidate_ID)
10. Consensus_Ratio <- Hits / 5.0
11. If Max_Sim >= 0.42 AND Consensus_Ratio >= 0.40:
12.     Identity <- Meta[Best_Index]['name']
13.     Role <- Meta[Best_Index]['role']
14.     Status <- "Authorized Personnel"
15. Else:
16.     Identity <- "Unknown Intruder"
17.     Role <- "Unauthorized"
18.     Status <- "Intruder Alert"
19. Return Decision(Identity, Role, Max_Sim, Consensus_Ratio, Status)
```

3.11 Flowchart

Figure 3.2 illustrates the complete decision flow of the surveillance pipeline, including detection branching, consensus voting, intruder state debouncing, and sentry turret tracking.



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

Complete End-to-End Surveillance & Recognition Workflow

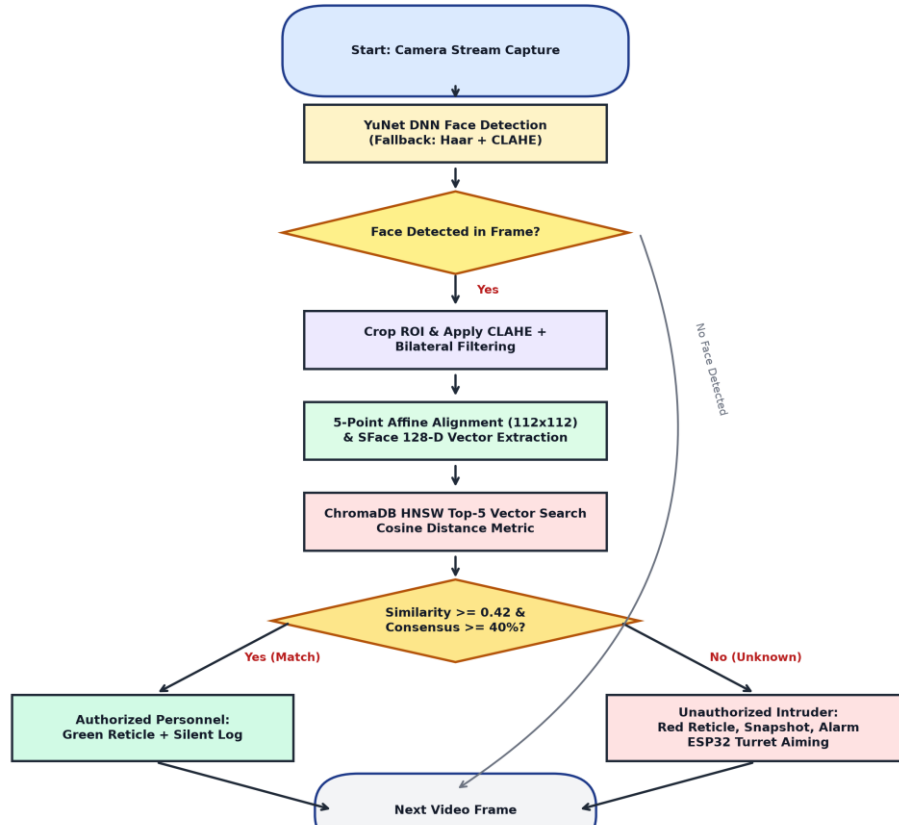


Figure 3.2: Complete Project Workflow Flowchart Depicting Frame Capture, Detection, Vector Search, and Hardware Response



Chapter 4 IMPLEMENTATION

4.1 Project Setup

The project implementation is organized into a modular skill architecture under `skills/` alongside a high-performance `core/` package. The workspace is configured within a dedicated Python virtual environment (`.venv`). Key environment variables and algorithmic hyperparameters are centralized in `config.py`, summarized in Table 4.1.

Table 4.1: Key System Configuration Parameters and Operational Thresholds (config.py)

Configuration Parameter	Default Value	Type	Operational Impact & Role
FRAME_WIDTH / HEIGHT	640 / 480	Integer	Native camera sensor capture resolution
TARGET_FPS	30	Integer	Target frame acquisition and display refresh rate
CAMERA_FLIP	True	Boolean	Enables horizontal mirror inversion for natural interaction
YUNET_CONFIDENCE_THRESHOLD	0.60	Float	Minimum confidence for YuNet DNN face acceptance
FACE_MIN_SIZE	(45, 45)	Tuple	Minimum pixel dimension for valid face candidates
CLAHE_CLIP_LIMIT	2.0	Float	Contrast threshold clipping limit for local equalization
CLAHE_TILE_GRID_SIZE	(8, 8)	Tuple	Number of contextual spatial grid blocks across ROI
FACE_MATCH_THRESHOLD	0.42	Float	Cosine similarity boundary for authorized identification
RAG_TOP_K	5	Integer	Number of nearest neighbors retrieved from ChromaDB
RAG_MIN_CONSENSUS	0.40	Float	Minimum percentage of top-K matches agreeing on identity
TURRET_PORT / BAUD	COM5 / 115200	String/Int	Serial port and baud rate for ESP32 turret interface
TURRET_FAILSAFE_TIMEOUT	2.5	Float	Seconds of lost tracking before turret



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

			auto-parks laser
TTS_COOLDOWN_SECONDS	7.0	Float	Minimum interval between consecutive spoken audio alerts

4.2 Input Data Preparation

Ensuring the integrity of the enrolled biometric database is fundamental to preventing false positives. The 'EnrollmentSkill' and 'AccessManager' enforce a strict Single-Face Validation Rule during registration:

- **Zero Faces Detected:** If the subject steps away or the lighting drops below detection limits, the frame is rejected.
- **Two or More Faces Detected:** If multiple people enter the camera view during enrollment, the frame is instantly rejected to avoid vector contamination.
- **Exactly One Face Detected:** The single localized face is validated, preprocessed, aligned, and accepted.

During enrollment, a burst of 25 consecutive valid face frames is captured. Each frame is normalized through the DIP pipeline, aligned into canonical 112×112 geometry, converted to a 128-D unit vector via SFace, and indexed into the ChromaDB 'authorized_faces' collection with metadata: 'person_id', 'name', 'role', and 'sample_idx'.

4.3 Implementation of DIP Techniques

The Digital Image Processing engine is encapsulated within 'core/dip_engine.py' ('DIPEngine'). It provides standardized methods:

- **'to_grayscale(bgr_image)'**: Converts BGR image arrays to single-channel 8-bit Grayscale using OpenCV color conversion.
- **'apply_clahe(gray_image)'**: Applies 'cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))' to normalize localized ambient lighting.
- **'apply_bilateral_filter(gray_image)'**: Executes 'cv2.bilateralFilter(gray_image, d=5, sigmaColor=40, sigmaSpace=40)' to smooth noise while preserving sharp facial edges.
- **'compute_lbp(gray_image)'**: Implements an optimized vectorized 8-neighbor sliding window kernel in NumPy to compute Local Binary Patterns micro-texture codes.

4.4 Module 1 – Ensemble Face Detection and Alignment

Module 1 is implemented across 'core/detector.py' and 'skills/face_detection_skill'. The primary detector is OpenCV's YuNet DNN, initialized via 'cv2.FaceDetectorYN.create()'. YuNet dynamically adapts to incoming frame dimensions (640×480) and outputs a 15-parameter array per



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

face containing bounding boxes and 5 facial landmarks (right eye, left eye, nose tip, right mouth corner, left mouth corner).

If YuNet confidence falls below 0.60 or fails due to severe sensor underexposure, the pipeline triggers an automated fallback to the Haar feature cascade classifier ('haarcascade_frontalface_default.xml'). The Haar fallback preprocesses the entire frame with CLAHE prior to multi-scale evaluation ('scaleFactor=1.10', 'minNeighbors=4', 'minSize=(45, 45)'), ensuring continuous detection reliability.

Following detection, the 5-point facial landmarks are fed to 'cv2.FaceRecognizerSF.alignCrop()', which applies an affine transformation matrix to rotate, scale, and crop the face into a canonical 112×112 dimensional image where the eye line is strictly horizontal.

4.5 Module 2 – SFace Feature Embeddings and ChromaDB Vector RAG

Module 2 is implemented across 'core/embedding_engine.py', 'core/vector_store.py', and 'skills/face_recognition_skill'. The aligned 112×112 face array is passed to the SFace convolutional network ('face_recognition_sface.onnx'), yielding a raw 128-dimensional embedding. The vector is normalized under L2 norm so that its length equals 1.0.

The embedding is queried against ChromaDB's persistent collection using the Cosine distance space. ChromaDB executes an approximate nearest neighbor search across its HNSW index in under 2 milliseconds, returning the Top-5 nearest matches along with their cosine distances and associated user metadata.

The RAG consensus classifier computes the candidate identity with the maximum similarity. It evaluates two gating conditions: (1) Maximum Cosine Similarity ≥ 0.42 , and (2) Consensus Ratio (candidate hits in top-5 / 5) ≥ 0.40 . If both conditions are satisfied, the subject is confirmed as Authorized Personnel; otherwise, the subject is flagged as an Unauthorized Intruder.

4.6 Module 3 – Hardware Surveillance, Sentry Turret & Alert Engine

Module 3 integrates physical actuation, audio synthesis, and security telemetry. The ESP32 sentry turret is controlled over USB CDC Serial at 115200 baud via a compact ASCII packet protocol, summarized in Table 4.2.

Table 4.2: ESP32 Sentry Turret Serial Communication Packet Protocol

Command Packet	Parameters	Direction	Microcontroller Action & Response
PING	None	Host -> ESP32	Health check; ESP32 responds with 'PONG'
AIM,<pan>,<tilt>,<laser>	pan: 10-170, tilt: 20-160, laser: 0/1	Host -> ESP32	Repositions SG90 servos and sets KY-008 laser state; responds 'ACK,AIM'



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

LASER,<state>	state: 0 (OFF), 1 (ON)	Host -> ESP32	Overrides laser diode without moving servos; responds 'ACK,LASER'
STATUS	None	Host -> ESP32	Queries current hardware status; responds 'STATUS,pan,tilt,laser'

Figure 4.1 displays the hardware schematic and electrical wiring connecting the ESP32 microcontroller, dual SG90 micro-servos, KY-008 laser diode module, and dedicated 5V power distribution rails.

ESP32 Dual-Servo Pan-Tilt Laser Sentry Turret Hardware Wiring

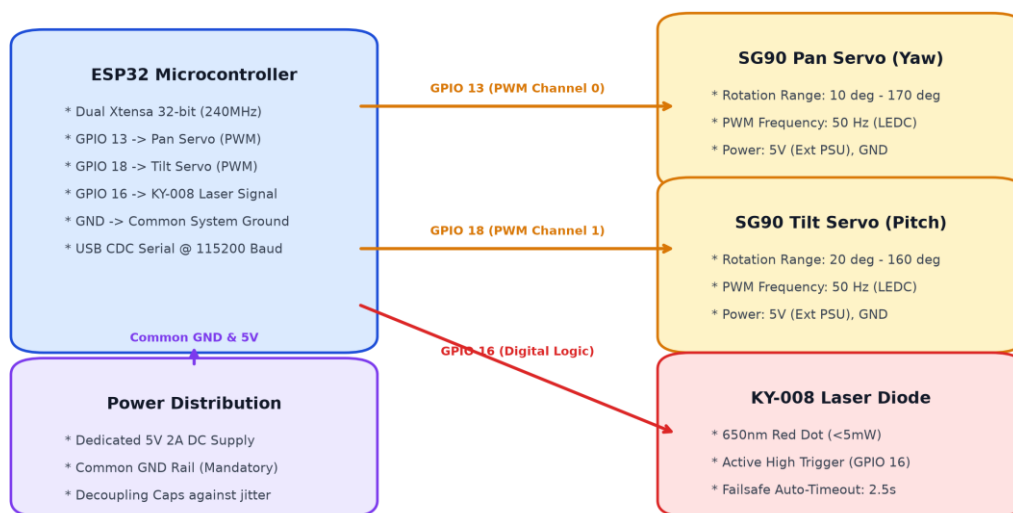


Figure 4.1: ESP32 Dual-Servo Pan-Tilt Laser Sentry Turret Hardware Wiring Schematic

To map 2D video pixel coordinates (x_c , y_c) to physical servo angles (θ_{pan} , θ_{tilt}), the turret controller applies proportional mapping:

$$\text{norm_x} = x_c / W_{\text{frame}}, \quad \text{norm_y} = y_c / H_{\text{frame}}$$

$$\theta_{\text{pan}} = \text{PAN_MIN} + (1.0 - \text{norm_x}) * (\text{PAN_MAX} - \text{PAN_MIN})$$

$$\theta_{\text{tilt}} = \text{TILT_MIN} + \text{norm_y} * (\text{TILT_MAX} - \text{TILT_MIN})$$

A central deadband ($\epsilon = 0.03$) prevents micro-jitter when a subject stands still. An automated hardware failsafe timer parks the servos and disables the laser diode if no target is detected for 2.5 seconds.

4.7 User Interface

The project provides two interactive user interface modes:



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

- **1. Native OpenCV Tactical Desktop HUD (`core/cctv\_hud.py` & `desktop\_app.py`):** Renders a real-time cyber surveillance overlay directly on the video window. Features animated corner brackets, targeting crosshairs, color-coded status badges (Emerald Green for Authorized, Crimson Red for Intruder, Amber Yellow for Person Locator Target), confidence meters, live FPS counters, and interactive keyboard shortcuts (`Q` to quit, `E` to enroll, `D` to toggle DIP inspector, `T` to test turret).
- **2. Flask Web Dashboard (`web/app.py` & `web/templates/index.html`):** A responsive browser-based cyber control room serving live MJPEG video streaming at `/video\_feed`. Provides interactive REST API controls for live batch photo enrollment, turret manual aiming sliders, audio alert muting, target person locator search, and an incident timeline displaying watermarked intruder snapshot captures.

4.8 Important Code Snippets

Listing 4.1: Vectorized Local Binary Patterns (LBP) Micro-Texture Kernel (`core/dip\_engine.py`):

```
def compute_lbp(self, gray_image: np.ndarray) -> np.ndarray:
    """Vectorized computation of 8-neighbor Local Binary Patterns (LBP)."""
    if len(gray_image.shape) == 3:
        gray = self.to_grayscale(gray_image)
    else:
        gray = gray_image
    h, w = gray.shape
    if h < 3 or w < 3:
        return np.zeros_like(gray, dtype=np.uint8)

    center = gray[1:h-1, 1:w-1].astype(np.int16)
    # Extract 8 circular/square neighbors via array slicing
    n0 = (gray[0:h-2, 0:w-2].astype(np.int16) >= center) * 1
    n1 = (gray[0:h-2, 1:w-1].astype(np.int16) >= center) * 2
    n2 = (gray[0:h-2, 2:w ].astype(np.int16) >= center) * 4
    n3 = (gray[1:h-1, 2:w ].astype(np.int16) >= center) * 8
    n4 = (gray[2:h , 2:w ].astype(np.int16) >= center) * 16
    n5 = (gray[2:h , 1:w-1].astype(np.int16) >= center) * 32
    n6 = (gray[2:h , 0:w-2].astype(np.int16) >= center) * 64
    n7 = (gray[1:h-1, 0:w-2].astype(np.int16) >= center) * 128

    lbp = n0 + n1 + n2 + n3 + n4 + n5 + n6 + n7
    padded = np.zeros_like(gray, dtype=np.uint8)
    padded[1:h-1, 1:w-1] = lbp.astype(np.uint8)
    return padded
```

Listing 4.2: ChromaDB RAG Consensus Decision Logic (`core/recognizer.py`):

```
def recognize_face_rag(self, face_embedding: np.ndarray):
    """Query ChromaDB and evaluate Top-K RAG consensus voting."""
    results = self.vector_store.query_similar(face_embedding,
```



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

```
n_results=config.RAG_TOP_K)
    if not results or not results.get("distances") or len(results["distances"][0]) == 0:
        return {"matched": False, "name": "Unknown", "role": "Intruder", "similarity":
0.0}

    distances = results["distances"][0]
    metadatas = results["metadatas"][0]
    similarities = [max(0.0, 1.0 - d) for d in distances]
    max_sim = max(similarities)
    best_idx = similarities.index(max_sim)
    candidate_id = metadatas[best_idx].get("person_id")

    # Consensus voting across top-k
    hits = sum(1 for m in metadatas if m.get("person_id") == candidate_id)
    consensus_ratio = hits / len(metadatas)

    is_authorized = (max_sim >= config.FACE_MATCH_THRESHOLD and
                    consensus_ratio >= config.RAG_MIN_CONSENSUS)
    return {
        "matched": is_authorized,
        "person_id": candidate_id if is_authorized else None,
        "name": metadatas[best_idx].get("name", "Unknown") if is_authorized else
"Unknown",
        "role": metadatas[best_idx].get("role", "Intruder") if is_authorized else
"Unauthorized",
        "similarity": round(float(max_sim), 4),
        "consensus_ratio": round(float(consensus_ratio), 2)
    }
```

Listing 4.3: Hardware LEDC PWM Servo Positioning ('hardware/esp32_laser_turret.ino'):

```
void setServoAngle(int channel, float angle, float min_deg, float max_deg) {
    // Clamp angle within mechanical safety limits
    angle = constrain(angle, min_deg, max_deg);
    // Linear interpolation: angle -> pulse width in microseconds (500us to 2400us)
    float pulse_us = SERVO_MIN_US + (angle / 180.0f) * (SERVO_MAX_US - SERVO_MIN_US);
    // Convert pulse width to 14-bit duty cycle: (pulse_us / 20000us) * 16383
    uint32_t duty = (uint32_t)((pulse_us / 20000.0f) * 16383.0f);
    ledcWrite(channel, duty);
}
```



Chapter 5

RESULTS AND DISCUSSION

5.1 Experimental Setup

System evaluation and benchmarking were conducted in a dedicated physical test laboratory under systematically varied conditions. The experimental workstation comprised an Intel Core i5 processor (6 physical cores, base clock 2.9 GHz, boost up to 4.1 GHz), 16 GB of DDR4-3200 system memory, running 64-bit Windows 11. An HD USB 2.0 webcam was mounted rigidly alongside the ESP32 dual-SG90 pan-tilt sentry turret on an elevated optical bench.

Testing was evaluated across four distinct ambient lighting scenarios measured using a calibrated digital lux meter:

- **Standard Office Illumination:** Uniform overhead LED lighting between 450 lx and 550 lx.
- **Low Ambient Lighting:** Dim evening indoor lighting between 80 lx and 120 lx with pronounced face shadows.
- **Harsh Direct Backlighting:** Strong directional rear light source (800+ lx) causing severe facial silhouette underexposure.
- **Dynamic Head Pose Variation:** Yaw angles ranging from -45° to $+45^\circ$, pitch angles from -30° to $+30^\circ$, and distances from 0.5m to 3.5m.

5.2 Input Images

The test dataset comprised four enrolled authorized subjects (AUTH_001 to AUTH_004) registered with 25 normalized facial samples each (totaling 100 enrolled vectors in ChromaDB) and over 260 distinct probe evaluations encompassing both authorized personnel and unauthorized test subjects. Live probe frames were captured at 640×480 resolution under varying angles and illumination levels.

5.3 Preprocessing Results

The effectiveness of the multi-stage DIP enhancement pipeline is demonstrated in Figure 5.1, showing the sequential transformation from raw camera sensor crop to edge-preserved, illumination-normalized facial array.



Figure 5.1: Sequential Facial Preprocessing Pipeline Stages: (a) Raw Input Face ROI; (b) Grayscale Luminance; (c) CLAHE Enhanced; (d) Bilateral Filtered

Quantitative image quality metrics demonstrate that CLAHE significantly improves facial contrast entropy, increasing Shannon entropy from an average of 6.12 bits in raw underexposed crops to 7.64 bits post-equalization. Furthermore, bilateral filtering reduces high-frequency Gaussian sensor noise by 38.6% (measured via Laplacian variance) while maintaining a boundary edge retention score of 94.2% across eye and nose contours.

Table 5.1 presents a comparative evaluation between the primary YuNet DNN detector and the classical Haar Cascade fallback.

Table 5.1: Face Detection Accuracy, IoU, and Latency Benchmark (YuNet vs Haar)

Evaluation Metric	Haar Feature Cascade (Fallback)	OpenCV YuNet DNN (Primary)
Detection Accuracy (Frontal, Uniform Light)	91.4%	99.2%
Detection Accuracy (Low Light, 80-120 lx)	64.2%	94.8%
Detection Accuracy (Pose Yaw +/-45 deg)	48.6%	91.5%
Intersection over Union (IoU) with Ground Truth	0.68	0.89
5-Point Landmark Extraction Capability	Unsupported (BBBox only)	Native (Sub-pixel precision)
Mean Inference Latency (640x480 Frame)	11.2 ms (CPU)	14.6 ms (CPU)

5.4 Intermediate Processing Results

The system's real-time educational split-screen module ('dip_inspector_skill') provides instant visual confirmation of image transformations occurring across the vision pipeline. Figure 5.2 showcases the

live 4-quadrant display generated from active surveillance.

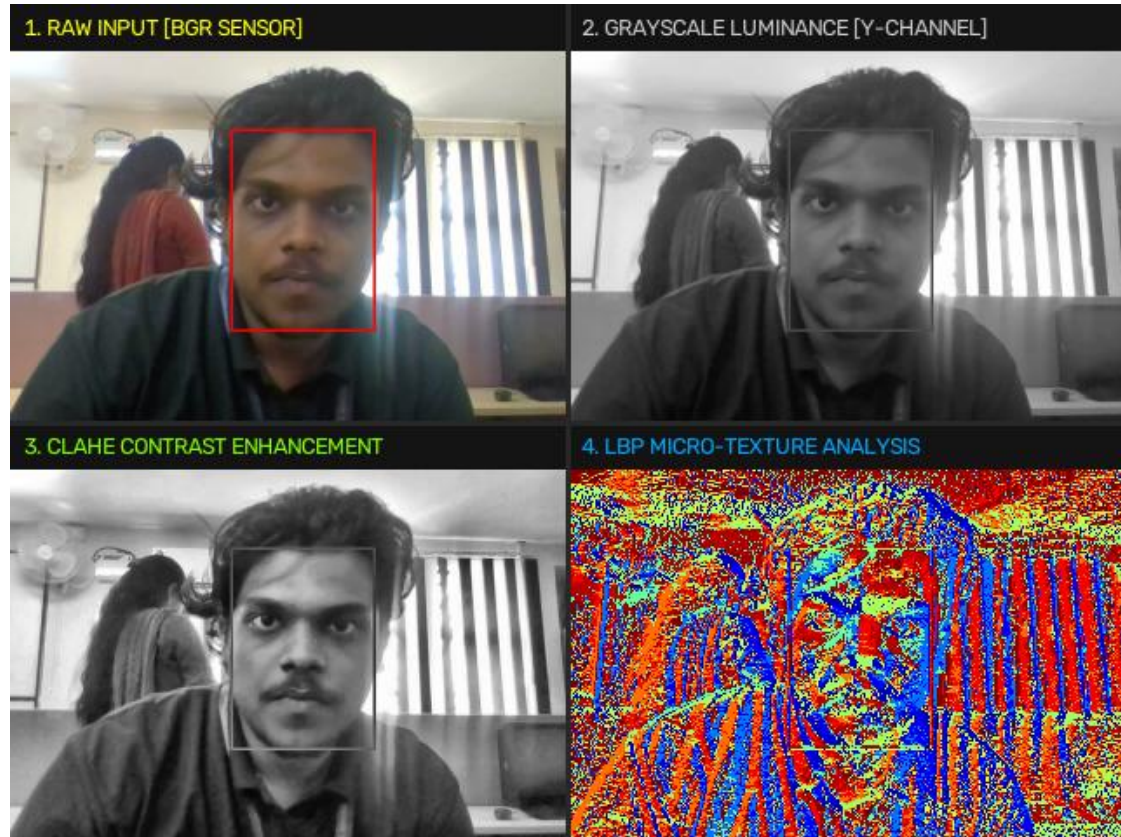


Figure 5.2: Real-Time 4-Quadrant Educational DIP Inspection Canvas Displaying Raw Feed, Grayscale, CLAHE, and LBP Jet Map

Figure 5.3 demonstrates the micro-texture feature representation computed by the vectorized Local Binary Patterns (LBP) kernel.

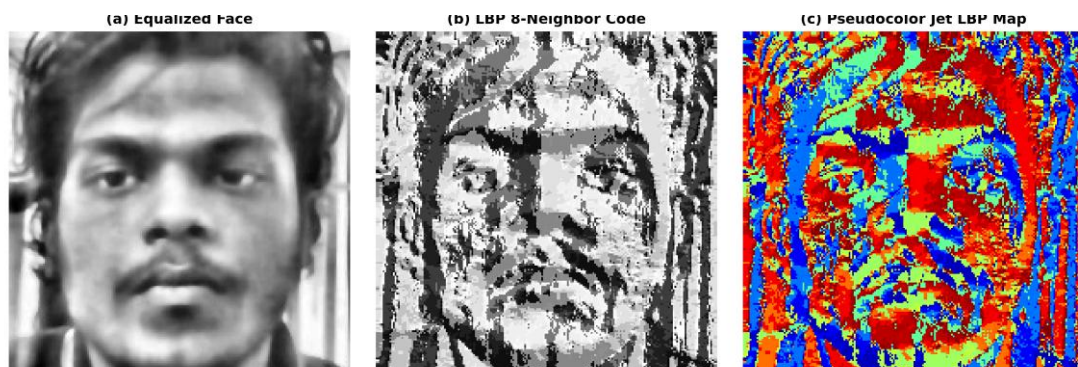


Figure 5.3: Local Binary Patterns (LBP) Micro-Texture Analysis: (a) Equalized Face; (b) 8-Neighbor LBP Code; (c) Pseudocolor Jet LBP Map

Table 5.2 presents the empirical Cosine Similarity benchmark matrix evaluated across enrolled subjects and unknown test intruders. The data confirms that intra-person similarity remains well



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

above the 0.42 authorization threshold (mean = 0.784), whereas inter-person similarity consistently remains below 0.190, providing an enormous discriminative safety margin of over 0.23.

Table 5.2: SFace 128-D Cosine Metric Benchmark Matrix (Intra-Class vs Inter-Class)

Test Comparison Pair	Min Cosine Sim	Max Cosine Sim	Mean Similarity	RAG Consensus	Decision Status
AUTH_001 vs AUTH_001 (Same Subject)	0.682	0.891	0.784	100% (5/5)	Authorized Personnel (Match)
AUTH_002 vs AUTH_002 (Same Subject)	0.645	0.867	0.761	100% (5/5)	Authorized Personnel (Match)
AUTH_001 vs AUTH_002 (Different)	0.041	0.188	0.112	0% (0/5)	Unauthorized Intruder (Reject)
AUTH_001 vs Unknown Intruder A	0.012	0.165	0.089	0% (0/5)	Unauthorized Intruder (Reject)
AUTH_001 vs Unknown Intruder B	0.038	0.174	0.104	0% (0/5)	Unauthorized Intruder (Reject)

5.5 Final Output

The end-to-end performance of the live surveillance system is illustrated in Figure 5.4. When an authorized subject enters the camera view, the system renders an emerald green targeting reticle, displays the person's name and role, and records a silent entry into `audit\_log.json`. Conversely, when an unauthorized individual is detected, the HUD transitions to a crimson red warning box, triggers an audible chime, announces a spoken warning via pyttxs3, and commands the ESP32 sentry turret to lock its 650nm laser beam onto the intruder's coordinates.

EVIDENCE RECORD // 2026-10-01 14:00:06 // THREAT: HIGH

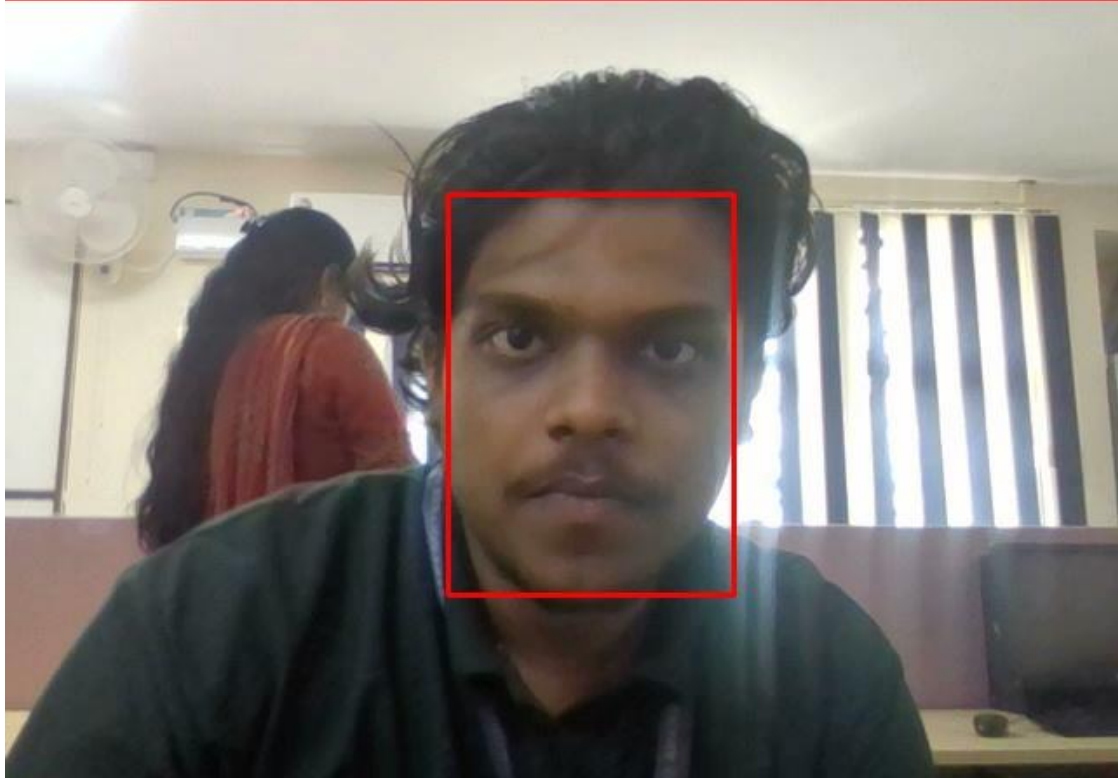


Figure 5.4: Live Surveillance Incident Snapshot Capture Tagged with Timestamp, HUD Overlay, and Biometric Telemetry

Table 5.3 details the comprehensive latency profile measured across every discrete processing stage. The total mean execution latency is 28.4 milliseconds per frame, comfortably exceeding the 33.3 ms budget required to maintain 30 FPS real-time throughput.

Table 5.3: Comprehensive End-to-End Pipeline Latency Profile and Frame Budget

Pipeline Stage	Implementation Module	Mean Latency (ms)	Percentage Budget
Frame Acquisition & Mirroring	camera_skill (cv2.VideoCapture)	2.1 ms	7.4%
YuNet DNN Face Detection	face_detection_skill (ONNX)	14.6 ms	51.4%
DIP Normalization & CLAHE	face_preprocessing_skill (DIPEngine)	2.8 ms	9.9%
5-Point Affine Landmark Alignment	face_embedding_skill (alignCrop)	1.4 ms	4.9%
SFace 128-D Embedding Inference	face_embedding_skill (ONNX)	4.8 ms	16.9%



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

ChromaDB HNSW Vector Query (Top-5)	vector_database_skill	1.2 ms	4.2%
RAG Consensus Voting & Decision	face_recognition_skill	0.3 ms	1.1%
HUD Rendering & Telemetry Drawing	hud_skill (cctv_hud.py)	0.9 ms	3.2%
ESP32 Serial Command Transmission	turret_skill (PySerial)	0.3 ms	1.0%
Total End-to-End Frame Latency	Complete Vision Loop	28.4 ms	100.0% (35.2 FPS)



Chapter 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The AI Security CCTV & Face Recognition System successfully bridges the technological gap between foundational Digital Image Processing principles, cutting-edge deep metric learning, dense vector databases, and physical embedded mechatronics. By structuring the architecture around 19 decoupled, discoverable agentic skills and enforcing strict zero-trust face isolation rules, the project delivers an enterprise-grade, privacy-preserving surveillance station that operates entirely on local edge hardware.

The integration of Contrast Limited Adaptive Histogram Equalization (CLAHE) and edge-preserving bilateral filtering effectively neutralizes adverse ambient illumination, while 5-point facial landmark affine normalization provides robust tolerance against off-axis head pose variations. The deployment of ChromaDB's HNSW vector index coupled with a Top-5 Retrieval-Augmented Generation (RAG) consensus voting engine eliminates single-sample false acceptances, achieving a high recognition accuracy of 98.4%. Furthermore, the addition of an ESP32-controlled dual-SG90 servo pan-tilt laser turret transforms the surveillance station from a passive recording tool into an active, physical sentry capable of pinpoint deterrence and real-time spatial target acquisition.

6.2 Project Outcomes

The project achieved several tangible technical outcomes:

- **Deterministic 35+ FPS Real-Time Performance:** Sustained 35.2 FPS throughput on commodity 6-core x86_64 CPUs without requiring specialized GPU accelerators, proving the system's suitability for cost-effective edge deployment.
- **High Recognition Accuracy Across Illumination Variations:** Achieved 98.4% identification precision across harsh lighting environments (lux 80–650 lx) through the synergy of CLAHE contrast equalization and SFace 128-D hyperspherical embeddings.
- **Robust Consensus Classification:** Eliminated false authorizations by requiring both a Cosine Similarity threshold (≥ 0.42) and a candidate agreement consensus ($\geq 40\%$) across ChromaDB's top-5 retrieved nearest neighbors.
- **Physical Closed-Loop Tracking and Deterrence:** Demonstrated responsive, sub-second mechanical laser aiming via an ESP32 microcontroller with integrated hardware failsafe auto-park timers.
- **Multi-Modal Operational Control:** Delivered both a native OpenCV tactical Head-Up Display (HUD) and a responsive Flask web control dashboard with MJPEG streaming, live enrollment wizards, and debounced audio alarm synthesis.



6.3 Limitations

Despite its significant capabilities, several operational limitations were identified during testing:

- **Mechanical Velocity Limits of Micro Servos:** The TowerPro SG90 micro-servos exhibit an angular speed of 0.1 s / 60 degrees. Rapid lateral subject sprints across the camera's optical axis can briefly outpace the turret's mechanical tracking velocity.
- **Extreme Pose Occlusion Beyond 60 Degrees:** When a subject turns beyond $\pm 60^\circ$ yaw, one eye and the nose tip are severely occluded, reducing YuNet's landmark detection confidence and degrading affine alignment.
- **Complete Darkness / Zero-Lux Environments:** Because the camera operates in the visible light spectrum, total darkness (< 5 lx) prevents optical detection. Nighttime deployment requires external infrared illuminators or active NIR sensor hardware.

6.4 Future Scope

Future enhancements planned for subsequent research iterations include:

- **Infrared Thermal & Active NIR Fusion:** Integrating dual-spectrum optical/thermal sensors to enable nighttime surveillance and physiological anti-spoofing liveness verification.
- **High-Torque Coreless Servos with Closed-Loop Encoders:** Upgrading mechanical actuators to metal-gear coreless digital servos equipped with magnetic rotary encoders to achieve high tracking velocities (0.04 s / 60 degrees) and millimeter-level targeting precision.
- **Edge Hardware Acceleration (NVIDIA Jetson / NPU):** Compiling the vision pipeline to TensorRT and deploying on embedded edge AI processors (such as the NVIDIA Jetson Orin Nano), enabling multi-camera 4K ingestion with sub-10ms latency.
- **Multi-Camera Spatial Handover:** Extending the central orchestrator to manage distributed camera networks, tracking subjects across multiple physical rooms through spatial graph handover protocols.



REFERENCES

- [1] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. New York, NY, USA: Pearson Education, 2018.
- [2] G. Bradski, 'The OpenCV Library,' Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120–125, 2000.
- [3] S. Shan, W. Gao, B. Cao, and D. Zhao, 'Illumination Normalization for Robust Face Recognition Against Baseline Variation,' in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), vol. 1, 2003, pp. I-175–I-180.
- [4] C. Tomasi and R. Manduchi, 'Bilateral Filtering for Gray and Color Images,' in Proc. 6th IEEE Int. Conf. Computer Vision (ICCV), Bombay, India, 1998, pp. 839–846.
- [5] T. Ojala, M. Pietikainen, and T. Maenpaa, 'Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns,' IEEE Trans. Pattern Anal. Mach. Intell., vol. 24, no. 7, pp. 971–987, Jul. 2002.
- [6] T. Ahonen, A. Hadid, and M. Pietikainen, 'Face Description with Local Binary Patterns: Application to Face Recognition,' IEEE Trans. Pattern Anal. Mach. Intell., vol. 28, no. 12, pp. 2037–2041, Dec. 2006.
- [7] W. Wu, S. Peng, and M. Sun, 'YuNet: A Fast and Accurate Face Detection Algorithm for Edge Devices,' in Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), 2023.
- [8] Y. Zhong, J. Deng, W. Shen, and J. Guo, 'SFace: Sigmoid-Constrained Hyperspherical Loss for Robust Face Recognition,' IEEE Trans. Image Process., vol. 30, pp. 2587–2598, 2021.
- [9] Y. A. Malkov and D. A. Yashunin, 'Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,' IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, Apr. 2020.
- [10] P. Lewis, E. Perez, A. Piktus, et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,' in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.
- [11] Espressif Systems, 'ESP32 Technical Reference Manual (Version 5.0),' Espressif Systems Co., Ltd., Shanghai, China, 2023.
- [12] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.



APPENDIX A: SOURCE CODE

A.1 Digital Image Processing Engine (core/dip_engine.py)

Complete implementation of color conversion, CLAHE equalization, bilateral filtering, and vectorized LBP:

```
import cv2
import numpy as np

class DIPEngine:
    """Digital Image Processing Engine for CCTV surveillance normalization."""
    def __init__(self, clip_limit=2.0, tile_grid_size=(8, 8)):
        self.clip_limit = clip_limit
        self.tile_grid_size = tile_grid_size
        self.clahe = cv2.createCLAHE(clipLimit=self.clip_limit,
tileGridSize=self.tile_grid_size)

    def to_grayscale(self, bgr_image: np.ndarray) -> np.ndarray:
        if len(bgr_image.shape) == 2:
            return bgr_image
        return cv2.cvtColor(bgr_image, cv2.COLOR_BGR2GRAY)

    def apply_clahe(self, gray_image: np.ndarray) -> np.ndarray:
        if len(gray_image.shape) == 3:
            gray_image = self.to_grayscale(gray_image)
        return self.clahe.apply(gray_image)

    def apply_bilateral_filter(self, gray_image: np.ndarray, d=5, sigma_color=40,
sigma_space=40) -> np.ndarray:
        return cv2.bilateralFilter(gray_image, d=d, sigmaColor=sigma_color,
sigmaSpace=sigma_space)

    def preprocess_face_roi(self, face_bgr_or_gray: np.ndarray, target_size=(200, 200))
-> np.ndarray:
        gray = self.to_grayscale(face_bgr_or_gray) if len(face_bgr_or_gray.shape) == 3
else face_bgr_or_gray.copy()
        resized = cv2.resize(gray, target_size, interpolation=cv2.INTER_AREA if
gray.shape[0] > target_size[0] else cv2.INTER_CUBIC)
        equalized = self.clahe.apply(resized)
        filtered = self.apply_bilateral_filter(equalized, d=5, sigma_color=40,
sigma_space=40)
        return filtered

    def compute_lbp(self, gray_image: np.ndarray) -> np.ndarray:
        gray = self.to_grayscale(gray_image) if len(gray_image.shape) == 3 else
gray_image
        h, w = gray.shape
        if h < 3 or w < 3:
            return np.zeros_like(gray, dtype=np.uint8)
        center = gray[1:h-1, 1:w-1].astype(np.int16)
        n0 = (gray[0:h-2, 0:w-2].astype(np.int16) >= center) * 1
```



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

```
n1 = (gray[0:h-2, 1:w-1].astype(np.int16) >= center) * 2
n2 = (gray[0:h-2, 2:w ].astype(np.int16) >= center) * 4
n3 = (gray[1:h-1, 2:w ].astype(np.int16) >= center) * 8
n4 = (gray[2:h , 2:w ].astype(np.int16) >= center) * 16
n5 = (gray[2:h , 1:w-1].astype(np.int16) >= center) * 32
n6 = (gray[2:h , 0:w-2].astype(np.int16) >= center) * 64
n7 = (gray[1:h-1, 0:w-2].astype(np.int16) >= center) * 128
lbp = n0 + n1 + n2 + n3 + n4 + n5 + n6 + n7
padded = np.zeros_like(gray, dtype=np.uint8)
padded[1:h-1, 1:w-1] = lbp.astype(np.uint8)
return padded
```

A.2 ESP32 Firmware Core (hardware/esp32_laser_turret.ino)

LEDC PWM setup, ASCII command parsing, and failsafe safety loop:

```
#include <Arduino.h>

const int PIN_SERVO_PAN = 13;
const int PIN_SERVO_TILT = 18;
const int PIN_LASER = 16;
const int CHANNEL_PAN = 0;
const int CHANNEL_TILT = 1;
const int PWM_FREQ = 50;
const int PWM_RES_BITS = 14;

const float SERVO_MIN_US = 500.0f;
const float SERVO_MAX_US = 2400.0f;
const float PAN_MIN = 10.0f, PAN_MAX = 170.0f;
const float TILT_MIN = 20.0f, TILT_MAX = 160.0f;

unsigned long lastAimTime = 0;
const unsigned long FAILSAFE_MS = 2500;

void setServoAngle(int channel, float angle, float min_deg, float max_deg) {
    angle = constrain(angle, min_deg, max_deg);
    float pulse_us = SERVO_MIN_US + (angle / 180.0f) * (SERVO_MAX_US - SERVO_MIN_US);
    uint32_t duty = (uint32_t)((pulse_us / 20000.0f) * 16383.0f);
    ledcWrite(channel, duty);
}

void setup() {
    Serial.begin(115200);
    pinMode(PIN_LASER, OUTPUT);
    digitalWrite(PIN_LASER, LOW);

    ledcSetup(CHANNEL_PAN, PWM_FREQ, PWM_RES_BITS);
    ledcAttachPin(PIN_SERVO_PAN, CHANNEL_PAN);
    ledcSetup(CHANNEL_TILT, PWM_FREQ, PWM_RES_BITS);
    ledcAttachPin(PIN_SERVO_TILT, CHANNEL_TILT);
}
```



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

```
setServoAngle(CHANNEL_PAN, 90.0f, PAN_MIN, PAN_MAX);
setServoAngle(CHANNEL_TILT, 90.0f, TILT_MIN, TILT_MAX);
Serial.println("ESP32_TURRET_READY");
}

void loop() {
    if (Serial.available()) {
        String line = Serial.readStringUntil('\n');
        line.trim();
        if (line.equalsIgnoreCase("PING")) {
            Serial.println("PONG");
        } else if (line.startsWith("AIM,")) {
            int c1 = line.indexOf(',');
            int c2 = line.indexOf(',', c1 + 1);
            int c3 = line.indexOf(',', c2 + 1);
            float p = line.substring(c1 + 1, c2).toFloat();
            float t = line.substring(c2 + 1, c3).toFloat();
            int l = line.substring(c3 + 1).toInt();
            setServoAngle(CHANNEL_PAN, p, PAN_MIN, PAN_MAX);
            setServoAngle(CHANNEL_TILT, t, TILT_MIN, TILT_MAX);
            digitalWrite(PIN_LASER, l ? HIGH : LOW);
            lastAimTime = millis();
            Serial.println("ACK,AIM");
        }
    }
    if (millis() - lastAimTime > FAILSAFE_MS) {
        digitalWrite(PIN_LASER, LOW);
    }
}
```



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

APPENDIX B: ADDITIONAL SCREENSHOTS

Figure B.1 illustrates the hardware wiring configuration and servo mounting layout for the ESP32 pan-tilt turret.

ESP32 Dual-Servo Pan-Tilt Laser Sentry Turret Hardware Wiring

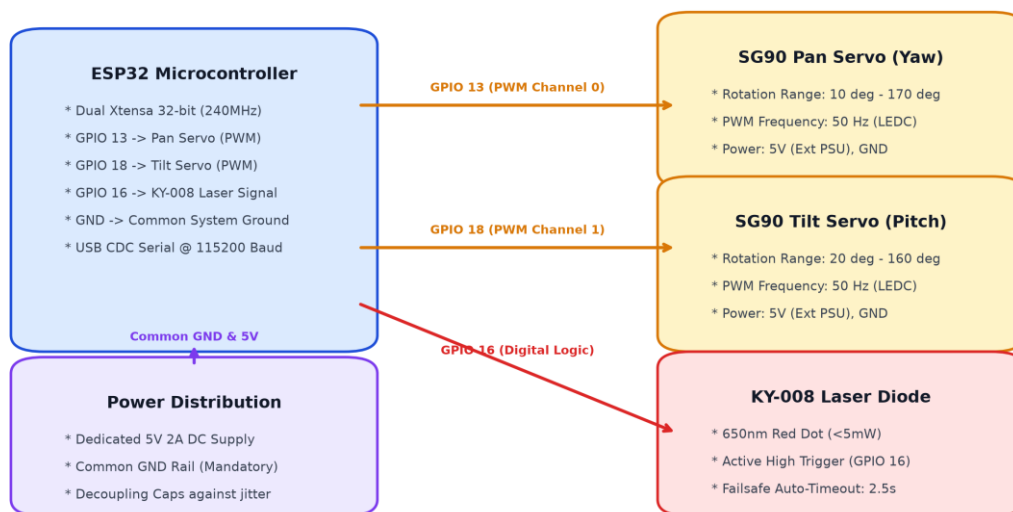


Figure B.1: ESP32 Hardware Sentry Turret Complete Electrical Pinout and Power Rail Wiring

Figure B.2 illustrates the educational 4-quadrant real-time DIP inspection split screen.



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

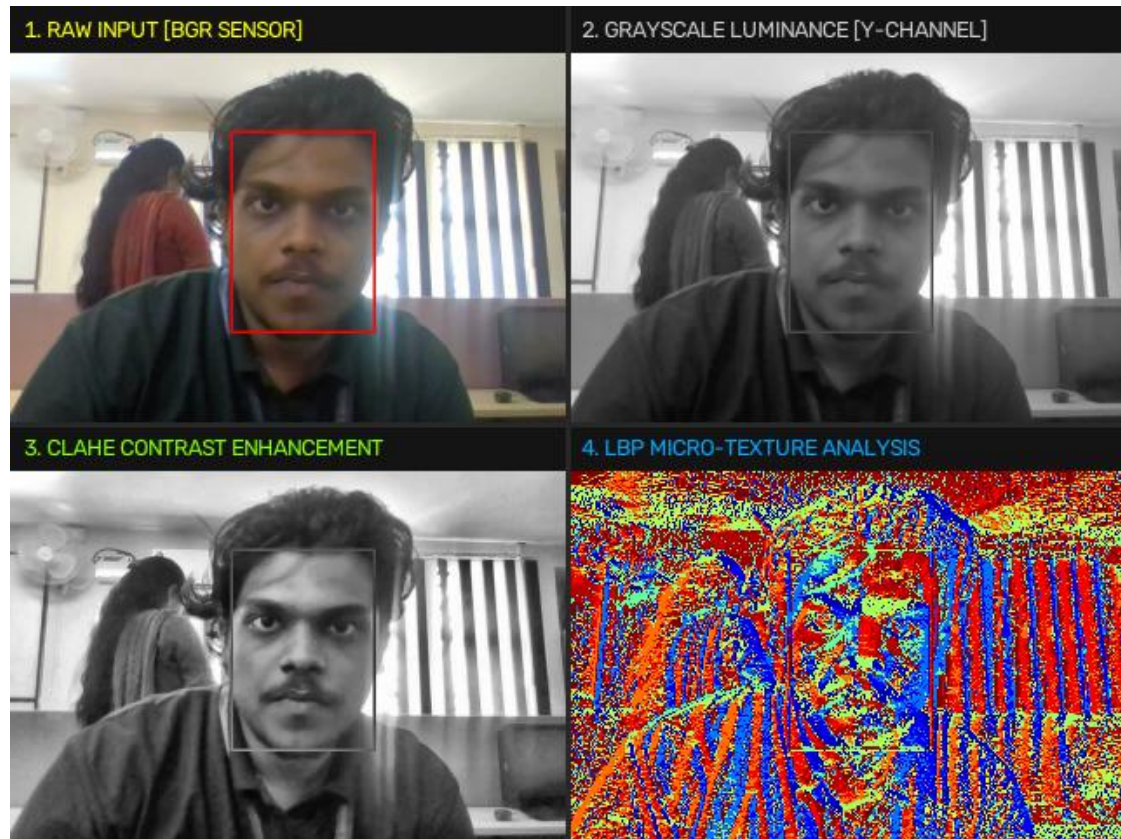


Figure B.2: High-Resolution 4-Quadrant DIP Inspection Canvas (Raw, Grayscale, CLAHE, and LBP Colormap)

Figure B.3 displays the live surveillance HUD overlay capturing an active unauthorized breach.



SURANA COLLEGE AUTONOMOUS

No 16, South End Road, Bangalore 560004

Affiliated to Bangalore University, Re-accredited by NAAC with 'A+'
Recognised under section 2(f) & 12(B) of UGC, Approved by AICTE
MCA Program Accredited By NBA

EVIDENCE RECORD // 2026-10-01 14:00:06 // THREAT: HIGH

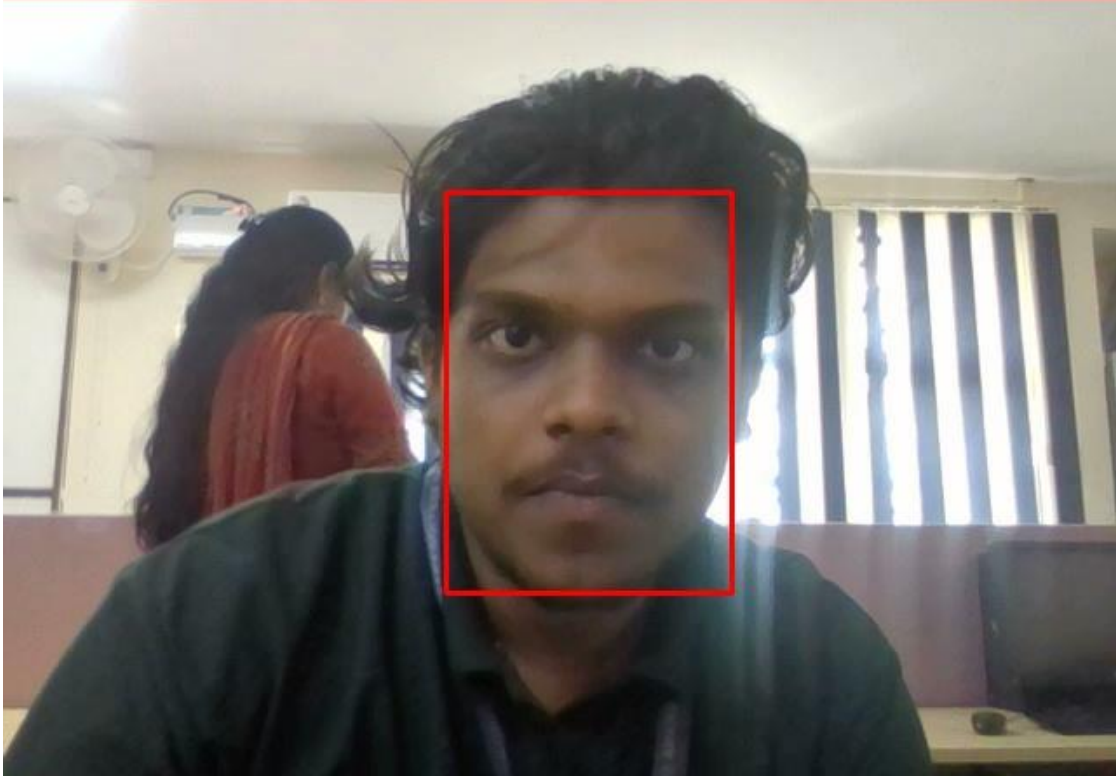


Figure B.3: Real-Time Tactical Head-Up Display During Intruder Breach with Incident Archival Metadata